

ASSIGNMENT 2

CMSC 451 (Section 0201, Fall 2024)

Due by 11:59 pm on Wednesday, October 2. Submit your solutions via Gradescope.

0. *Collaboration.*

Indicate that you understand the course collaboration policy and have followed it when working on this assignment. List the other students with whom you discussed the problems, or else indicate that you did not discuss any problems with your classmates.

If you consulted any resources other than the textbook to help you understand how to approach a problem, list them at the end of the solution of that problem.

I understand the course collaboration policy, and I collaborated with Omkar Pathak, Jason Zhong, and Shril Shah.

1. *Gears*. [20 points]

Suppose you are given a configuration of n circular gears. Specifically, for each pair of gears, you are told whether or not their teeth mesh, so that turning one would cause the other to turn. One of the gears has a crank on it, and you are told which one. Design an efficient algorithm that determines (i) whether it is possible to turn the crank, and if so, (ii) whether *all* the gears move when the crank is turned. Prove that your algorithm is correct and determine its asymptotic running time as a function of n . For full credit, your algorithm should have the smallest possible asymptotic running time.

Algorithm. Let us convert the context of gears to an undirected graph such that we can use traditional graph theory terminology. Let each gear be a vertex. Two vertices u, v have an edge if gears the gears u and v 's teeth mesh. The key insight in this problem is to realize that if two nodes are connected (ie. have an edge), the the gears must rotate in opposite directions. That is, if gear u rotates clockwise and it's teeth mesh with gear v , then gear v must rotate counterclockwise. In graph terms, to show (i), we must show if the graph is bipartite. To show (ii), we must show that the graph is bipartite and all n nodes in the graph are visited.

CheckGears(arr, gear)

// Assume input **arr** format is [(u, v, bool), ...] and **gear** is the gear with the crank

Let **G** be an empty adjacency list graph representation

// In **directions**, $-1 \rightarrow$ not yet visited, $0 \rightarrow$ counterclockwise, $1 \rightarrow$ clockwise

Set **directions** to a list of size n , initialized to -1 s

For **u**, **v**, **bool** in **arr**

If **bool** == **True**

Append **u** to **G[v]** and **v** to **G[u]**

// Start a BFS from **gear**

Let **q** be a queue with the element (**gear**, 0) // Start with direction 0, WLOG

While **q** is not empty

 Set **node** and **dir** be the "active" node and directions popped from the end of the **q**

If **directions[node]** == -1 // this gear hasn't been visited yet

 Set **directions[node]** = **dir**

For each **neighbor** of gear **node** // Neighbors must be direction $1 - \text{dir}$

 Add the element (**neighbor**, $1 - \text{dir}$) to the **q**

If **directions[node]** != **dir**

Return (**False**, **False**)

For **node** in **n**

If **directions[node]** == -1

Return (**True**, **False**)

Return (**True**, **True**)

Proof of Correctness. The algorithm converts the given "edge list" similar graph representation to an adjacency list graph representation. We then start a BFS starting from the gear with the crank, and without loss of generality, allow it to spin in direction 0, which I define to be counterclockwise. All neighbors of the "active" vertex during the BFS are then appended to the queue with the opposite direction. This direction in the queue must be enforced, given the fixed, existing direction of the predecessor node. If later on in the BFS, we deque one of these elements from the queue and find that it is assigned the opposite direction in **directions** than the assigned direction in **q**, then the graph cannot be bipartite, and so we return (**False, False**). If the BFS completes, then we can turn the crank. To determine if all of the gears turn, we simply have to check that all gears are visited in this one BFS, so iterate through **directions** and check if any elements are -1. If yes, we return (**True, False**) else we return (**True, True**).

Time Complexity.

In this list **arr** filled with 3-tuples, there are $\binom{n}{2} = \frac{n!}{(n-2)!2!} = n(n-1)/2 = \frac{1}{2}n^2 - \frac{n}{2} \implies \mathcal{O}(n^2)$. The BFS is $\mathcal{O}(V + E)$, and checking if all gears were visited is $\mathcal{O}(V)$. Thus, the overall time complexity of this algorithm is $\mathcal{O}(n^2)$.

2. *Touring a DAG.* [20 points]

Given a directed acyclic graph $G = (V, E)$, your goal is to determine whether there is a *tour* of G , i.e., a path that visits each vertex exactly once. If there is such a path, your algorithm should find it. Prove that your algorithm is correct and analyze its asymptotic running time as a function of $n = |V|$ and $m = |E|$. For full credit, your algorithm should run in time $O(m + n)$.

Algorithm.

CheckTour(V , E)

Set T to **TopologicalSort**(V , E)

Set G be a $V \times V$ adjacency matrix initialized with 0s

For u , v in E

Set $G[u][v] = 1$

Set $i = 0$

While $i < V - 1$

If $G[T[i]][T[i+1]] \neq 1$

Return (**False**, **NULL**)

Increment $i = i + 1$

Return (**True**, T)

Proof of Correctness and Time Complexity. Since the graph G is a DAG, it has a topological sorting that contains all vertices in V . This ordering ensures that vertices are sorted based on their dependencies to other vertices. After getting the topological ordering T , **CheckTour** will iterate through all adjacence elements in T and check if there exists an edge between the two vertices. If this is true, then we return a tuple containing **True** and the topological ordering T , otherwise we return **False** and **NULL**. The time complexity of **CheckTour** is guaranteed to be $O(m + n)$ because topological sort is $O(m + n)$, setting up the adjacency matrix is $O(m)$, and the while loop is $O(n)$ because checking if there's an edge between two nodes given an adjacency matrix is $O(1)$. Summing all these up, we get a running time of $O(m + n)$, as desired. $\square$

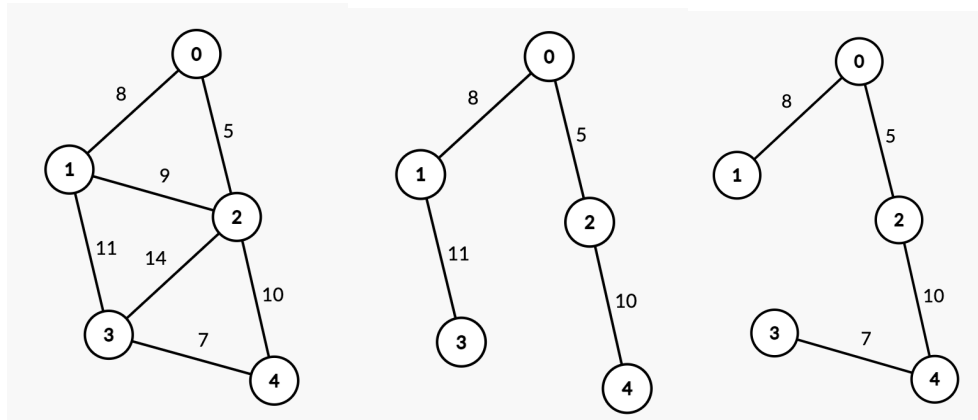
3. *Dijkstra vs. Prim.*

Dijkstra's algorithm for finding a shortest-path tree and Prim's algorithm for finding a minimum spanning tree both effectively build the tree by adding low-weight edges, but they have different goals and need not produce the same output.

- [5 points] Give an example of a weighted, undirected graph G with a designated vertex s such that Dijkstra's algorithm on G starting from s and Prim's algorithm on G starting from s produce different results. (Try to make your example as simple as possible.)
- [15 points] Suppose that after running Prim's algorithm on $G = (V, E)$ with starting vertex s and producing a minimum spanning tree T , you would like to check whether T is also the shortest path tree in G starting from s . You could do this using Dijkstra's algorithm, but that would take time $O((m + n) \log n)$, where $n = |V|$ and $m = |E|$. Show (by giving an explicit algorithm) that in fact it is possible to determine whether T is the shortest path tree in only $O(m + n)$ time. Your algorithm can take as input G (including its edge weights), s , and T , where T is represented by specifying the parent of each vertex. Prove both that your algorithm is correct and that its running time is $O(m + n)$. You can assume that all edge weights and shortest path lengths are distinct.

Solution.

- From left to right: the original graph, the shortest-path tree, and the minimum spanning tree, both generated from the source node 0.



- Consider the following algorithm.

Algorithm.

ShortestPathsFromMST(G, s, T)

Let dist be a vertex-indexed array storing shortest distances from s using edges in T all initially initialized as -1

Set $\text{dist}[s] = 0$

Let q be a queue initialized with the starting vertex s

While q isn't empty

Let u be the "active" vertex popped from q

For each neighbor v of u with weight w

```

    If  $T[v] = u$ 
        Set  $\text{dist}[v] = \text{dist}[u] + w$ 
        Push  $v$  into  $q$ 
    Return  $v$ 

```

CheckShortest(G, s, T)

```

    Set  $\text{dist} = \text{ShortestPathsFromMST}(G, s, T)$ 
    Let  $\text{dist2}$  be a vertex-indexed array storing shortest distances from  $s$  using edges in
     $G$ , all initially initialized as  $-1$ 
    Let  $q2$  be a queue initialized with the starting vertex  $s$ 
    Set  $\text{dist2}[s] = 0$ 
    While  $q2$  isn't empty
        Pop a vertex  $u$  from  $q2$ 
        For each neighbor  $v$  of  $u$  with weight  $w$ 
            If  $w + \text{dist2}[u] < \text{dist}[v]$ 
                Return False
            Else if  $w + \text{dist2}[u] = \text{dist}[v]$ 
                Set  $\text{dist2}[v] = \text{dist}[v]$ 
                Push  $v$  to  $q2$ 
    Return True

```

Proof of Correctness. Note that the function **ShortestPathsFromMST** correctly produces the shortest path from s to all other vertices in the MST, T . By definition of a tree, there only exists one path from any two nodes. Therefore, for each node $\neq s$, since it is the only path in the MST, it is also the shortest. The function **CheckShortest** correctly determines if the MST is also the shortest-path tree. By the above, dist holds the shortest distances from s to all other nodes using edges in the MST. dist2 will be similar, but can use all edges in G . The algorithm begins a BFS traversal starting at s . Now, while traversing, we can check distances in the MST with dist . While traversing, if we encounter a path from s to some other node that is shorter than the shortest path found in the MST, then we return **False** because therefore, the MST cannot be the shortest-path tree of G . At the end of the traversal, if we have visited all vertices and not exited the function by returning **False**, then we have essentially just followed the MST, which means the MST is the shortest-path tree and the algorithm returns **True**, as desired.

Time Complexity. Creating dist and dist2 take $\mathcal{O}(n)$ time. **ShortestPathsFromMST** essentially does a BFS and visited all nodes and edges, so this takes $\mathcal{O}(n + m)$ time. **CheckShortest** essentially does the same, so that also takes $\mathcal{O}(n + m)$ time, and vertex-lookup is always constant time because dist and dist2 are vertex-indexed arrays. Therefore, the overall time complexity is $\mathcal{O}(n + m)$ time, as desired.

4. *Scheduling to maximize profit.* [20 points]

Suppose you are given n jobs, where job i has deadline d_i (a positive integer) and profit p_i (a positive real number) for all $i \in \{1, 2, \dots, n\}$. Each job takes unit time, so it may be scheduled in any interval $[s_i, s_i + 1]$ for some nonnegative integer s_i provided $s_i + 1 \leq d_i$ (i.e., the job is finished by its deadline) and no other job is scheduled during the same interval. Your goal is to schedule a subset of the jobs to maximize the total profit, which is the sum of p_i over all scheduled jobs i . Design a greedy algorithm for this problem. Prove that your algorithm finds an optimal schedule, and analyze its running time.

Algorithm.

MaximizeProfit(deadlines, profits)

Sort the jobs in descending order s.t. $p_1 \geq p_2 \geq \dots \geq p_n$ // Compute $d_{\max}$ along the way

Set **S** be a list of size $\min(n, d_{\max})$ initialized to -1s

Let **free** be the size $\min(n, d_{\max})$ list $[1, 2, \dots, \min(n, d_{\max})]$

For each job j in 1 to n

Set $m = \min(n, d_j)$ // Get the latest free slot $\leq \min(n, d_j)$

If $m > 0$

Set $S[m] = j$ // Schedule j there

// Update **free**

$m' = m$

While $S[m'] \neq -1$

$\text{free}[m'] = \text{free}[m - 1]$

Increment $m' = m' + 1$

Proof of Correctness. *Greedy Scheduler Maximizes Profits.* Let J be the output from the greedy algorithm, and let S_J be the feasible schedule of jobs J . Similarly, let I be the output from another "optimal" algorithm, and S_I is the feasible schedule of jobs I . Note that one can rearrange $S_J \rightarrow S'_J$ and $S_I \rightarrow S'_I$ such that any $j \in I \cap J$ is done in the same slot in S'_J and S'_I . For all jobs $s \in J \cap I$

- (a) If s in the same slot in S_J and S_I , continue
- (b) If S_J schedules s earlier than S_I , in S_J , swap s with t (where s is scheduled in S_I). This is still feasible in S_J .
- (c) If S_J schedules s later than S_I , in S_J do a similar swap as (b). Still feasible.

Once a job is aligned in S'_J and S'_I , it never needs to be moved again, and we notably have that

$$\text{profit}(S_J) = \text{profit}(S'_J) \qquad \text{profit}(S_I) = \text{profit}(S'_I)$$

If $S'_J \neq S'_I$, this can only happen if $J \neq I$, in which we can only have a few cases:

- (a) Some job $s \in S'_J$ is opposite an empty slot in S'_I and $s \notin I$. This contradicts the fact that I is optimal, since s is feasible, by definition of the J , the greedy set of jobs, and adding s to S'_I would make it more profitable.
- (b) Some job $s \in S'_I$ is opposite an empty slot in S'_J and is feasible and more profitable. Therefore, $J \cup \{s\}$ is still feasible, but the greedy algorithm wouldn't have skipped it.
- (c) WLOG, suppose there's some job $s \in J$ and $t \in I$ in the same slot. If $p_s > p_t$, this contradicts the optimality of I . If $p_t > p_s$, the greedy algorithm skipped over t to eventually choose s . This contradicts the greediness of the algorithm. Therefore, $p_s = p_t$ is the only possibility, and so therefore

$$\text{profit}(S'_J) = \text{profit}(S'_I)$$

and so the greedy algorithm outputs the optimal (most profitable) scheduling.

Time Complexity. Sorting is $\mathcal{O}(n \log n)$, then iterating through the jobs and for each iteration, potentially updating **free** is worst case $\mathcal{O}(n^2)$. Therefore, the overall time complexity of this algorithm is $\mathcal{O}(n^2)$.

5. *Counting significant inversions.* [20 points]

Given a list of numbers $(a_1, a_2, \dots, a_n)$, a *significant inversion* is a pair of indices $i < j$ such that $a_i > 2a_j$. Design a divide-and-conquer algorithm to count the number of significant inversions in a list of length n . Prove that your algorithm is correct and analyze its running time. For full credit, your algorithm should run in time $O(n \log n)$.

Algorithm.

MergeSort(arr: List, sorted: List, left: int, right: int)

Initialize invCount = 0

If left < right

Set mid = floor((left + right) / 2)

Add invCount += MergeSort(arr, sorted, left, mid)

Add invCount += MergeSort(arr, sorted, mid + 1, right)

Add invCount += Merge(arr, sorted, left, right, mid + 1)

Return invCount

Merge(arr: List, sorted: List, left: int, right: int, mid: int)

Set invCount = 0

Set i = left, j = mid, k = right // Count inversions

While i ≤ mid - 1 and j ≤ right

If arr[i] > 2 * arr[j]

Add invCount += (mid - i)

Increment j

Else increment i

Reset i = left, j = mid, k = right // Merge the sorted arrays

While i ≤ mid - 1 and j ≤ right

If arr[i] < arr[j]

Set sorted[k] = arr[i]

Increment i and k

Else

Set sorted[k] = arr[j]

Increment j and k

// Copy any remaining elements of both subarrays to sorted

While i ≤ mid - 1

Set sorted[k] = arr[i]

Increment i and k

While j ≤ right

```

    Set sorted[k] = arr[j]
    Increment j and k
// Copy elements back to the original
For i from left to right + 1
    Set arr[i] = sorted[i]
Return invCount

```

CountInversions(arr: List)

Let sorted be a list of length n, initialized to all 0s

Return MergeSort(arr, sorted, 0, n - 1)

Proof of Correctness. We will take the vanilla **MergeSort** and **Merge** algorithms as already proven and only prove that the modifications in some functions yield an overall function **CountInversions** that correctly counts the number of inversions. We will proceed with proof by induction.

Base Case: Assume **arr** is of length 1. **MergeSort** is called and returns 0, which is correct.

Inductive Hypothesis: Assume that **CountInversions** works correctly for list sizes 1 to n

Inductive Step: Note that the length of the left and right subarrays are in $[1, n]$, and so by the Inductive Hypothesis, **invCount** after the second "Add **invCount** += ..." line is correct. The only case that we have to consider now is the number of significant inversions where **i** is the left half of the array and **j** is in the right half of the array, as all other cases are covered. This is in the first **While** loop in **Merge**, which follows a two pointer method. Note that both halves of the array are sorted by properties of **MergeSort**. If $\text{arr}[\text{i}] > 2 * \text{arr}[\text{j}]$, then all elements from **i** to **mid** are also significant inversions with **j**, and so we add $\text{mid} - \text{i}$ to **invCount**. In this case, we increment **j** to get a larger number in the right half of the array. Once we reach a large enough **j** such that $\text{arr}[\text{i}] \not> 2 * \text{arr}[\text{j}]$, we increment **i**, as there is no point incrementing **j** because getting a larger $\text{arr}[\text{j}]$ will not make this inequality True. This loop terminates either **i** reaches **mid** or **j** reaches **right**, and so every necessary pair has been processed, and **invCount** correctly equals the number of significant inversions for a list of length $n + 1$.

Time Complexity. The algorithm recursively divides lists into two halves to count the number of significant inversions. Note that both counting the number of inversions and merging takes $\mathcal{O}(n)$ time. Therefore, the recurrence relation for this algorithm can be modeled as

$$F(n) = 2F\left(\frac{n}{2}\right) + \mathcal{O}(2n)$$

By the Master Theorem, the time complexity for this algorithm is therefore $\mathcal{O}(n \log n)$.